

Setup and characterizing the solution

We want to iterate backward on the Bellman equation

$$V_t(s, a) = \max_{c, a'} u(c) + \beta \mathbb{E}[V_{t+1}(s', a') | s] \quad (1)$$

$$\text{s.t. } a' + c = (1 + r_t)a + y_t(s) \quad (2)$$

$$a' \geq \underline{a} \quad (3)$$

We will sometimes refer to the right side of (2) as *cash on hand*, $\text{coh}_t(s, a) \equiv (1 + r_t)a + y_t(s)$.

Denote the time-varying optimum of the problem (1), as a function of s and a , by $c_t(s, a)$ and $a'_t(s, a)$. It is characterized by an envelope condition on a

$$V_{a,t}(s, a) = (1 + r_t)u'(c_t(s, a)) \quad (4)$$

and a first-order condition on a'

$$u'(c_t(s, a)) \geq \beta \mathbb{E}[V_{a,t+1}(s', a'_t(s, a)) | s] \quad (5)$$

where we have equality if the borrowing constraint (3) does not bind.

Note that only the derivative V_a with respect to assets, not the level, of the value function appears in (4) and (5). Our goal will be, given this derivative $V_{a,t+1}$ at time $t + 1$, to obtain the consumption and asset policy functions a'_t and c_t at time t , and also the derivative $V_{a,t}$.

Discussion of algorithm

Suppose that there are discretely many $s \in \mathbf{S}$, and further that we are given a finite grid $\mathbf{A} = (a_0, \dots, a_{n_a-1})$ that discretizes the space of possible assets a . Now suppose that we are given $V_{a,t+1}(s, a)$ for all gridpoints $(s, a) \in \mathbf{S} \times \mathbf{A}$. How do we get $a'_t(s, a)$, $c_t(s, a)$, and $V_{a,t}(s, a)$, all also for every point $(s, a) \in \mathbf{S} \times \mathbf{A}$?

Discounting and expectations. Define the “end-of-period” value function W_t as

$$W_t(s, a') \equiv \beta \mathbb{E}[V_{t+1}(s', a') | s] \quad (6)$$

The function W_t gives the discounted expected value of V_{t+1} at the end of period t , once the asset choice a' has already been made, but before the new realization of the new exogenous state s' in period $t + 1$. With this definition, (5) simplifies to

$$u'(c_t(s, a)) \geq W_{a,t}(s, a'_t(s, a)) \quad (7)$$

If we have $V_{a,t+1}(s', a')$ for all gridpoints $(s', a') \in \mathbf{S} \times \mathbf{A}$, our first step is to obtain $W_{a,t}(s, a')$ for all $(s, a') \in \mathbf{S} \times \mathbf{A}$. This part is simple: we write $W_{a,t}(s, a') = \beta \sum_{s'} \pi_{ss'} V_{a,t+1}(s', a')$, where $\pi_{ss'}$ is the Markov probability of moving from state s to state s' . In matrix terms, if $\mathbf{W}_a$ and $\mathbf{V}_a$ are the $|\mathbf{S}| \times |\mathbf{A}|$ matrices containing $W_{a,t}(s, a')$ and $V_{a,t+1}(s', a')$, and $\Pi = [\pi_{ss'}]$ is the matrix of Markov probabilities, this can be written simply as

$$\mathbf{W}_a = \beta \Pi \mathbf{V}_a \quad (8)$$

Solving for asset policy using the first-order condition. For now, assume that (7) holds with equality. We will abuse notation slightly by defining, for each s , $a_t(s, a')$ to be the inverse of $a'_t(s, a)$, i.e. $a_t(s, a'_t(s, a)) \equiv a$. Then (7) can be rewritten as

$$c_t(s, a_t(s, a')) = \underbrace{(u')^{-1}(W_{a,t}(s, a'))}_{\equiv c_t^{endog}(s, a')} \quad (9)$$

We can compute the right side of (9) for every point $(s, a') \in \mathbf{S} \times \mathbf{A}$ on our grid. On the left, we have time- t consumption at the so-called *endogenous gridpoint* $(s, a_t(s, a'))$, which is not (except by coincidence) a point on the actual grid. The endogenous gridpoint is whatever gridpoint leads us to pick assets a' for tomorrow. We denote consumption there, as a function of a' , by $c_t^{endog}(s, a')$.

Let's substitute (9) for c_t in the budget constraint (2). Rearranging, this gives us

$$(1 + r_t)a_t(s, a') + y_t(s) = c_t^{endog}(s, a') + a' \quad (10)$$

from which we can isolate $a_t(s, a')$ as

$$a_t(s, a') = \frac{c_t^{endog}(s, a') + a' - y_t(s)}{1 + r_t} \quad (11)$$

What do we have now? We can calculate the right side of (11), and therefore $a_t(s, a')$, for every (s, a') on the grid. But we want to know the inverse, $a'_t(s, a)$, for every (s, a) on the grid instead.

How do we get this? Take any s as given. As a function of a , $a'_t(s, a)$ passes through the points $(a_t(s, a'), a')$. We can linearly interpolate these points to get the value of $a'_t(s, a)$ for each a on the grid.¹

An optional improvement. There is one optional tweak that is convenient in practice, and that we will use as our preferred approach. Linear interpolation is invariant to an affine transformation of the x -axis. In particular, suppose that we do the transformation $a \mapsto (1 + r_t)a + y_t(s)$. Then our data points are $((1 + r_t)a_t(s, a') + y_t(s) + a', a') = (c_t^{endog}(s, a') + a', a')$, and we linearly interpolate to find the values of this function at $coh_t(s, a) \equiv (1 + r_t)a + y_t(s)$ for each $a \in \mathbf{A}$. This will give us the exact same values of $a'_t(s, a)$ as above. But it is slightly more efficient and concise to describe, since it involves only objects that we must calculate at some point anyway: c_t^{endog} and coh_t .

Enforcing the borrowing constraint and backing out consumption. We just calculated $a'_t(s, a)$ assuming that the first-order condition (7) held with equality. But if the borrowing constraint (3) is binding, this need not be the case.

Fortunately, this is easy to address. Wherever our calculated policy satisfies the constraint $a'_t(s, a) \geq \underline{a}$, it is the correct policy. Whenever it does not satisfy the constraint, i.e. whenever $a'_t(s, a) < \underline{a}$, we simply set $a'_t(s, a) = \underline{a}$.²

Once we have enforced the borrowing constraint $\underline{a}$, we then back out consumption from the budget constraint (2): $c_t(s, a) = coh_t(s, a) - a'_t(s, a)$.

¹We could use another interpolation method instead, like cubic spline interpolation or interpolation with Chebyshev polynomials. But with finitely many s and the occasionally binding constraint $a' \geq \underline{a}$, the policy functions generally have kinks (are not continuously differentiable), so that these higher-order interpolation methods do not necessarily perform any better than linear interpolation.

²One can formally prove that this approach is valid by using the convexity of the optimization problem.

Using the envelope condition to recover the derivative of the value function. Given the policy function $c_t(s, a)$, we finally apply the envelope condition (4) to get $V_{a,t}(s, a) = (1 + r_t)u'(c_t(s, a))$.

Summary of algorithm

In each iteration, we start with $V_{a,t+1}(s', a')$ at all gridpoints $(s', a') \in \mathbf{S} \times \mathbf{A}$, and aim to calculate the time- t policy functions $a'_t(s, a)$ and $c_t(s, a)$, along with $V_{a,t}(s, a)$, at all gridpoints $(s, a) \in \mathbf{S} \times \mathbf{A}$.

Step 1. Calculate $W_{a,t}(s, a') \equiv \beta \mathbb{E}[V_{a,t+1}(s', a')|s]$ for all gridpoints $(s, a') \in \mathbf{S} \times \mathbf{A}$.

To do this, given an $|\mathbf{S}| \times |\mathbf{A}|$ matrix $\mathbf{V}_a$ containing all $V_{a,t+1}(s', a')$, calculate $\mathbf{W}_a = \beta \Pi \mathbf{V}_a$ where $\Pi = [\pi_{s,s'}]$ is the Markov transition matrix. The resulting matrix $\mathbf{W}_a$ contains all $W_{a,t}(s, a')$.

Step 2. Calculate $c_t^{endog}(s, a') \equiv (u')^{-1}(W_{a,t}(s, a'))$.

Then, for each $s \in \mathbf{S}$, linearly interpolate each point $coh_t(s, a)$ (for all $a \in \mathbf{A}$) against the data points $\{(c_t^{endog}(s, a'), a')\}_{a' \in \mathbf{A}}$ to obtain the policy $a'_t(s, a)$.

Step 3. Enforce the borrowing constraint $a'_t(s, a) \geq \underline{a}$ by replacing $a'_t(s, a)$ with $\max(a'_t(s, a), \underline{a})$. Then compute $c_t(s, a) = coh_t(s, a) - a'_t(s, a)$.

Step 4. Compute $V_{a,t}(s, a) = (1 + r_t)u'(c_t(s, a))$.